

Exercise Sheet 4

Projections and Clipping

Question 1.

In OpenGL, perspective projections are implemented through perspective normalization.

- (a) (2 points) Show that in OpenGL the transformation matrix for a perspective projection, defined by the viewing volume with the parameters *left*, *right*, *top*, *bottom*, *near*, and *far*, has the following form:

$$P = \begin{pmatrix} \frac{2 \cdot \text{near}}{\text{right} - \text{left}} & 0 & \frac{\text{right} + \text{left}}{\text{right} - \text{left}} & 0 \\ 0 & \frac{2 \cdot \text{near}}{\text{top} - \text{bottom}} & \frac{\text{top} + \text{bottom}}{\text{top} - \text{bottom}} & 0 \\ 0 & 0 & \frac{-(\text{far} + \text{near})}{\text{far} - \text{near}} & \frac{-2 \cdot \text{far} \cdot \text{near}}{\text{far} - \text{near}} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Consider which transformation steps are required for perspective projection.

- (b) (3 points) Additionally, compute the inverse matrix of the transformation matrix.
- (c) (3 points) The following matrix is given:

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -3 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}$$

The matrix corresponds to the result of multiplying the projection matrix with the orientation matrix.

Calculate the direction and upward vector of the camera, as well as the distance between near and far plane.

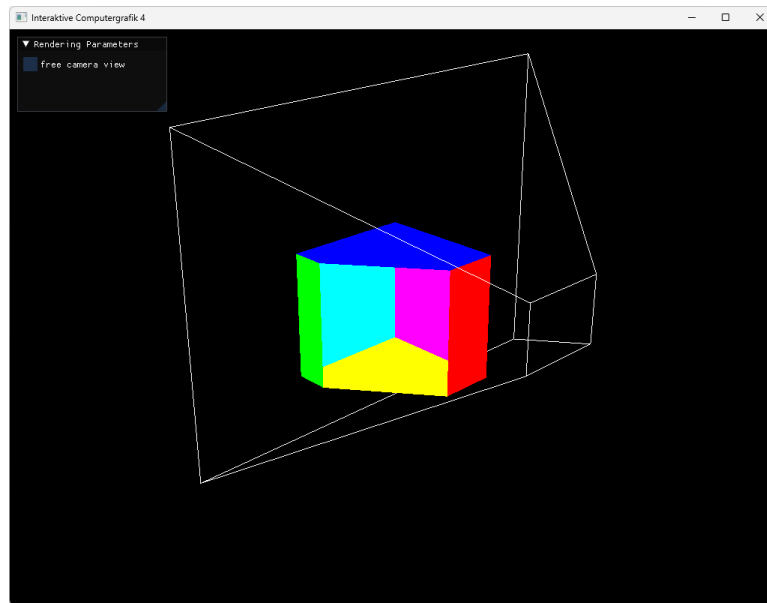
Question 2.

With this exercise sheet, a framework is available for download in which programming tasks are to be solved. The framework includes a second camera model (**free_camera**) that can be moved in the *z* and *x* directions using the 'W', 'A', 'S' und 'D' keys.

This camera is not initially used for display but serves only as a tool to implement clipping. The camera used for display can be operated as usual. In the GUI, you can switch between cameras using the *freecameraview* toggle.

- (a) (2 points) Edit the **clipVertices** method in the **main.cpp** file and extract the 6 planes of the view frustum from the **mvp** matrix into **frustumPlanes**. The planes should be represented in Hesse normal form (normal vector in *x*-, *y*-, and *z*-components of each vector, distance to the origin in the *w*-component).
- (b) (10 points) Implement the Sutherland-Hodgeman algorithm in the **clipVertices** function and clip all triangles in **vertices** against the planes of the view frustum. Store the vertices of the resulting triangles in **clippedVertices**.

If the clipping works, it should look like the following picture (Backface culling is intentionally disabled to see the inside of the cube):



All sections where you need to modify the source code are marked with a comment in the form of **//**
TODO:.

Submit your solution by **Monday, June 30, 2025, at 23:55** via the Moodle platform in the form of a ZIP file.